14. ROS2 DDS

1. Introduction to DDS

DDS stands for Data Distribution Service, that is, the data distribution service. It was released and maintained by the Object Management Group (OMG) in 2004. It is a data distribution/subscription standard designed specifically for real-time systems. It was first applied in the US Navy to solve the compatibility problem of large-scale software upgrades in the complex network environment of ships. Now it has become a mandatory standard.

DDS emphasizes being data-centric and can provide rich quality of service policies to ensure that data is distributed in real time, efficiently, and flexibly, satisfying the needs of various distributed real-time communication applications.

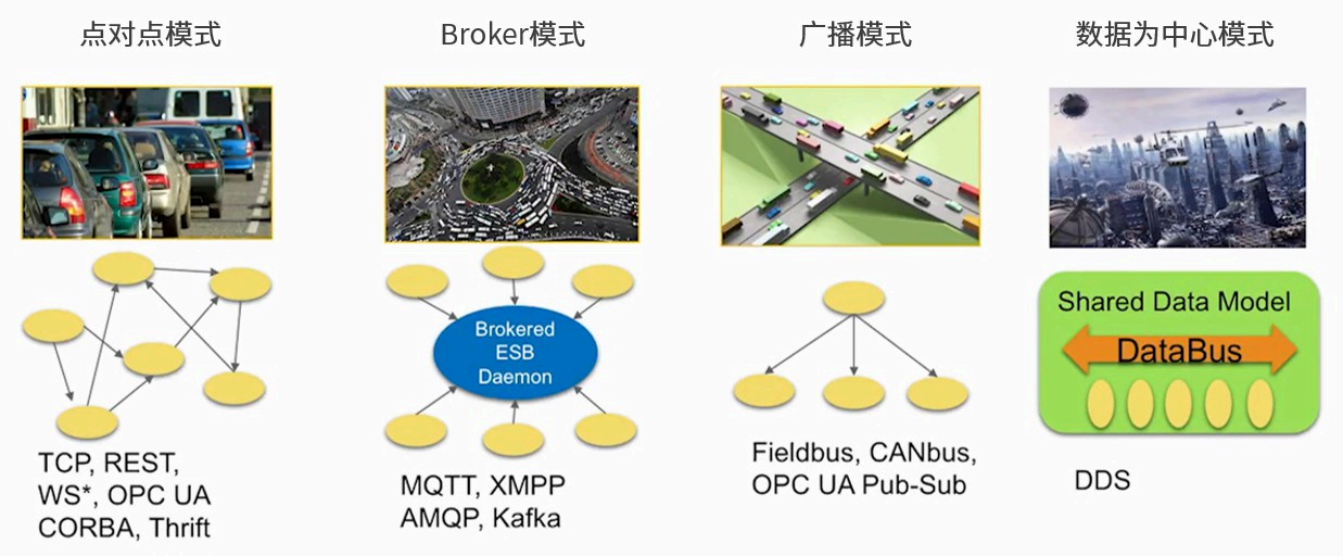
References:

- Fast DDS official documentation: 3.3.0
- ROS2 official documentation on advanced DDS features: Click to jump

2. Communication Model

The topics, services, and actions we learned about in the previous courses all rely on DDS for their specific underlying communication implementation. DDS is like the neural network in the ROS robot system.

The core of DDS is communication. There are many models and software frameworks capable of realizing communication; here we list the four commonly used models.



- The first is the **peer-to-peer model**, in which many clients connect to one server. Each communication requires a connection to be established between the two parties. As the number of communicating nodes increases, the number of connections also increases. Moreover, each client needs to know the specific address of the server and the services it provides; once the server address changes, all clients are affected.
- The second is the **Broker model**, which optimizes the peer-to-peer model. The Broker centrally processes everyone's requests and further locates the role that can actually respond to the service. In this way, the client does not need to care about the specific address of the server. However, the

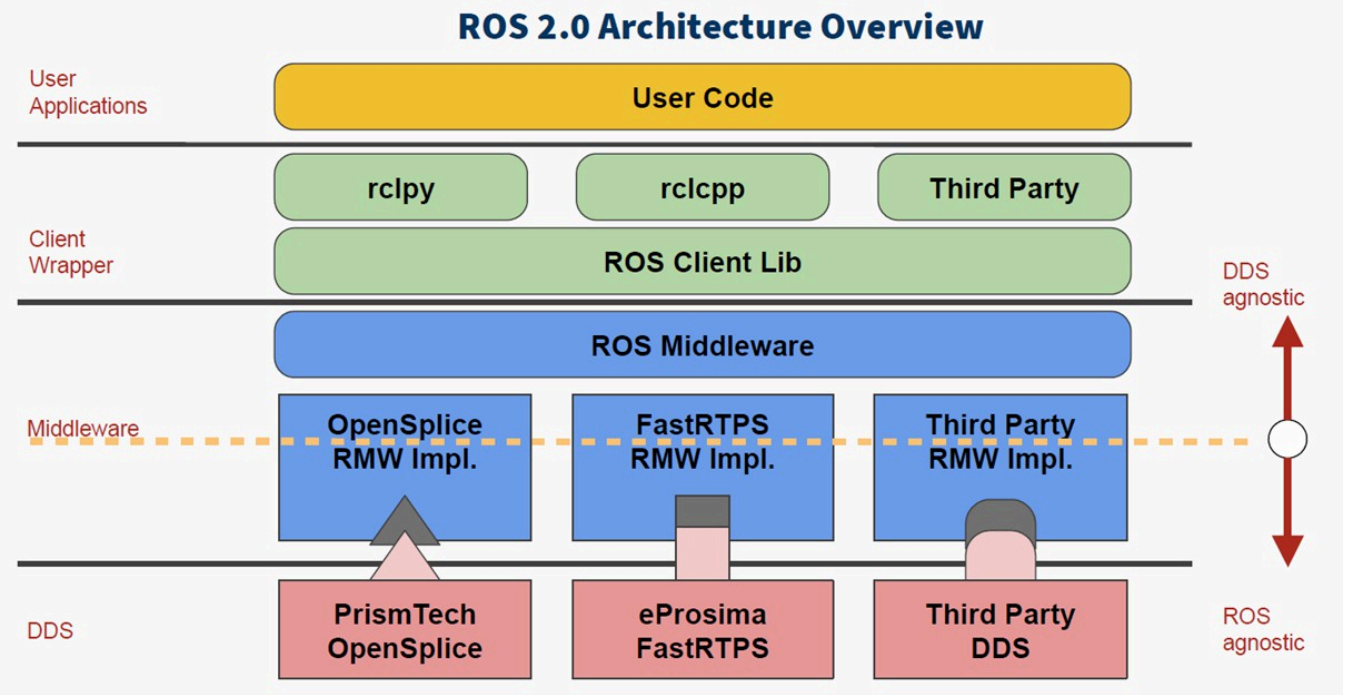
problem is also obvious: the Broker is the core, and its processing speed affects the efficiency of all nodes. When the system scale grows to a certain level, the Broker becomes the performance bottleneck of the whole system. More troublesome still, if the Broker fails, the entire system may fail to operate normally. The earlier ROS1 system used an architecture similar to this.

- The third is the **broadcast model**, in which all nodes can broadcast messages on the channel and all nodes can receive the messages. This model solves the problem of the server address, and the communicating parties do not need to establish a separate connection either. However, there are too many messages on the broadcast channel, and all nodes must care about every message, although many of them have nothing to do with the node itself.
- The fourth is the **data-centric DDS model**. This model is somewhat similar to the broadcast model, in which all nodes can publish and subscribe to messages on the DataBus. Its advanced part lies in the fact that there are many parallel channels in the communication, and each node can care only about the messages it is interested in and ignore the ones it is not, a bit like a rotating hot pot where all the delicious food is transferred on this DataBus and we only need to take what we want to eat; the rest has nothing to do with us.

It can be seen that among these communication models, DDS has more obvious advantages.

3. Application of DDS in ROS2

DDS occupies a crucial position in the ROS2 system, and all the upper-layer constructions are built on top of DDS. In this ROS2 architecture diagram, the blue and red parts are DDS.



Among the four major components of ROS, the addition of DDS greatly improves the overall capability of the distributed communication system. In this way, during robot development we no longer need to worry about communication problems and can spend more time on the application development of the other parts.

4. Quality of Service Policy QoS

The basic structure in DDS is the Domain, which binds various applications together for communication. Recall the DOMAIN ID we configured when setting up communication between the Raspberry Pi and the computer: it is the grouping definition of the global data space. Only the nodes in the same DOMAIN group can communicate with each other. This avoids resources being occupied by useless data.

Another important feature of DDS is the quality of service policy: QoS.

QoS is a network transmission policy. The application specifies the network transmission quality behavior it needs, and the QoS service implements the requirements of this behavior, satisfying the customer's demand for communication quality as much as possible. It can be understood as a contract between the data provider and the receiver.

The policies are as follows:

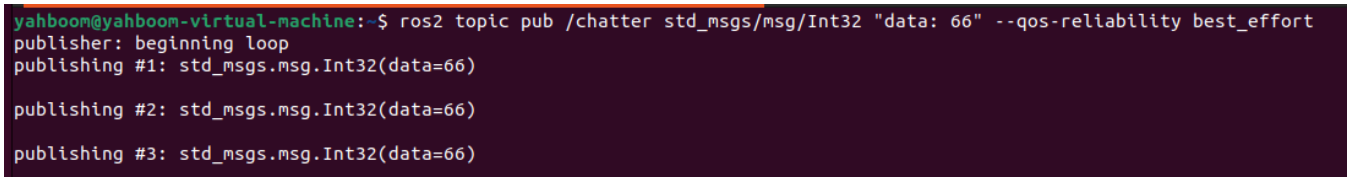
- The **DEADLINE** policy means that the communication data must complete one communication within each deadline;
- The **HISTORY** policy means the cache size for historical data;
- The **RELIABILITY** policy means the data communication mode. Configured as BEST_EFFORT, it is the best-effort transmission mode: even when the network condition is poor, smooth data transmission is guaranteed, which may result in data loss. Configured as RELIABLE, it is the reliable mode, which can try its best to guarantee the integrity of the images during communication. We can choose the appropriate communication mode according to the functional scenario of the application;
- The **DURABILITY** policy can be configured for late-joining nodes, and it also guarantees that a certain amount of historical data is sent over, so that new nodes can quickly adapt to the system.

5. Test Cases

5.1 Case 1—Configure DDS through the Command Line

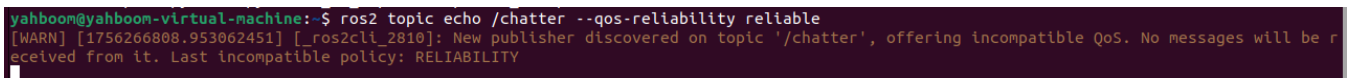
- Open the first terminal and publish the topic with the following command:

```
ros2 topic pub /chatter std_msgs/msg/Int32 "data: 66" --qos-reliability best_effort
```



- Open another terminal and print the topic using a different QoS. If the QoS policy used is different from that of the publisher, a warning message will appear and the topic data cannot be received properly:

```
ros2 topic echo /chatter --qos-reliability reliable
```



- Using the same QoS policy as the topic publisher allows us to receive the topic data

```
ros2 topic echo /chatter --qos-reliability best_effort
```

```
yahboom@yahboom-virtual-machine:~$ ros2 topic echo /chatter --qos-reliability best_effort
data: 66
---
data: 66
```

5.2 Case 2—Write a Topic Node and Configure the QoS Service Policy

- Use the following command to create a new function package

```
ros2 pkg create learning_dds --build-type ament_python --dependencies rclpy std_msgs
```

- Create a new dds_controller_pub.py file as the publisher for topic communication, and fill in the following content:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
# Import QoS related classes
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class ControllerPublisher(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure the QoS policy: reliable transmission, keep the last 1 historical
data
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.RELIABLE,  # Reliable transmission (retransmit
lost data)
            history=QoSHistoryPolicy.KEEP_LAST,         # Keep the last N pieces of data
            depth=1                                     # Keep 1 historical data
        )
        # 2. Create the publisher: topic name /robot_cmd, message type String, QoS policy
        self.publisher = self.create_publisher(
            String,
            "/robot_cmd",
            self.qos_profile
        )
        # 3. Create the timer: send one command per second
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.cmd_list = ["forward", "backward", "stop"]  # Command list
        self.cmd_index = 0  # Command index, switching in a loop


    def timer_callback(self):
        # Cycle through the commands (forward → backward → stop → forward...)
        current_cmd = self.cmd_list[self.cmd_index % 3]
        # Create the message and fill in the data
        msg = String()
        msg.data = current_cmd
        # Publish the message
```

```
        self.publisher.publish(msg)
        # Print the log (shows the published command)
        self.get_logger().info(f"Publish control command: {msg.data}")
        # Update the command index
        self.cmd_index += 1


def main(args=None):
    # Initialize ROS2
    rclpy.init(args=args)
    # Create the publisher node
    node = ControllerPublisher("robot_controller_pub")
    # Run the node in a loop
    rclpy.spin(node)
    # Destroy the node and shut down ROS2
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

- The subscriber code implementation: create a new dds_robot_sub.py file and fill in the following subscriber implementation code

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class RobotSubscriber(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure a QoS policy compatible with the publisher
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1
        )
        # 2. Create the subscriber: topic name /robot_cmd, callback function, QoS policy
        self.subscription = self.create_subscription(
            String,
            "/robot_cmd",
            self.cmd_callback,  # Callback function executed after data is received
            self.qos_profile
        )

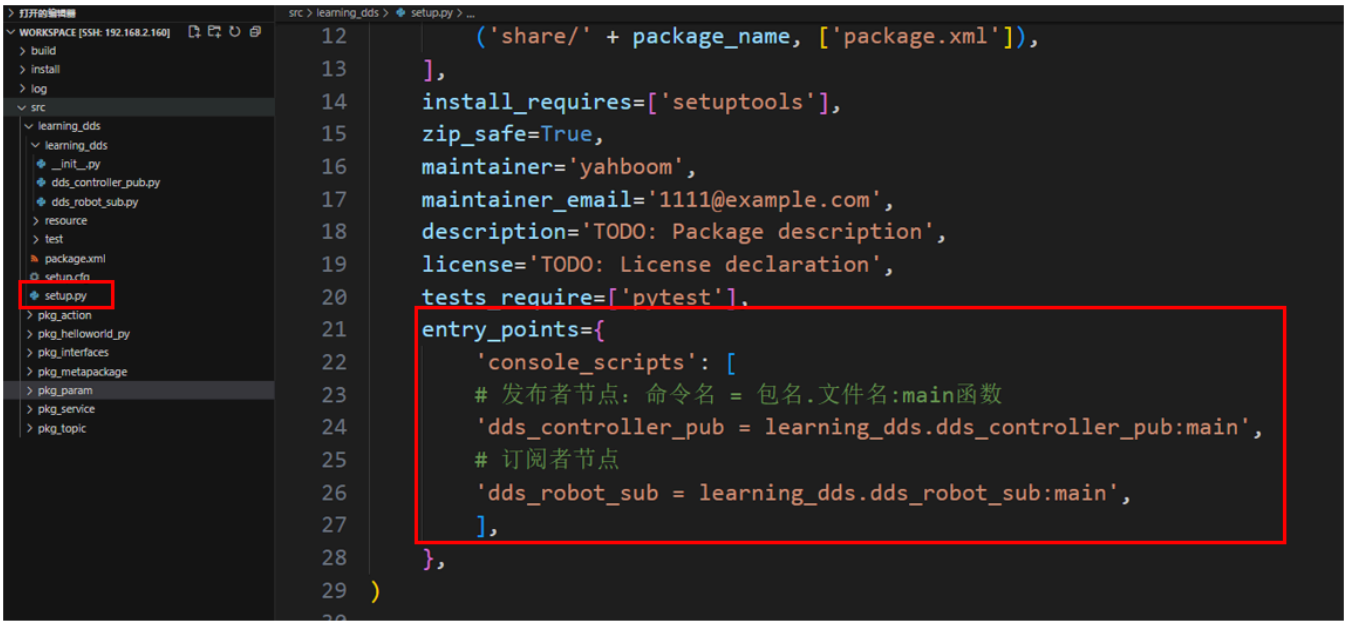
    def cmd_callback(self, msg):
        # Callback function: process the received command
        self.get_logger().info(f"Received control command: {msg.data} → execute the
corresponding action")
```

```
def main(args=None):
    rclpy.init(args=args)
    node = RobotSubscriber("robot_subscriber")
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

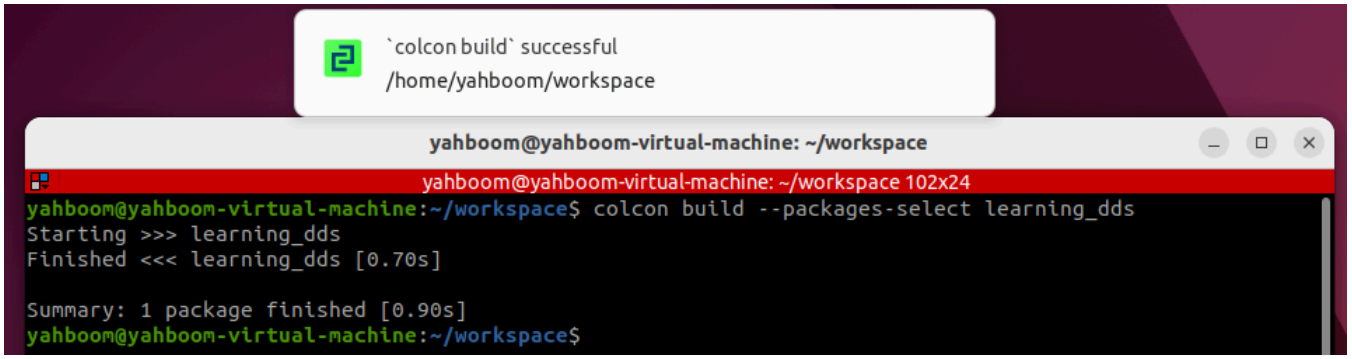
- Configure the compilation file (setup.py)

```
entry_points={
    'console_scripts': [
        # Publisher node: command name = package name . file name : main function
        'dds_controller_pub = learning_dds.dds_controller_pub:main',
        # Subscriber node
        'dds_robot_sub = learning_dds.dds_robot_sub:main',
    ],
},
```



- Open a terminal in the workspace directory and compile the function package

```
colcon build --packages-select learning_dds
```



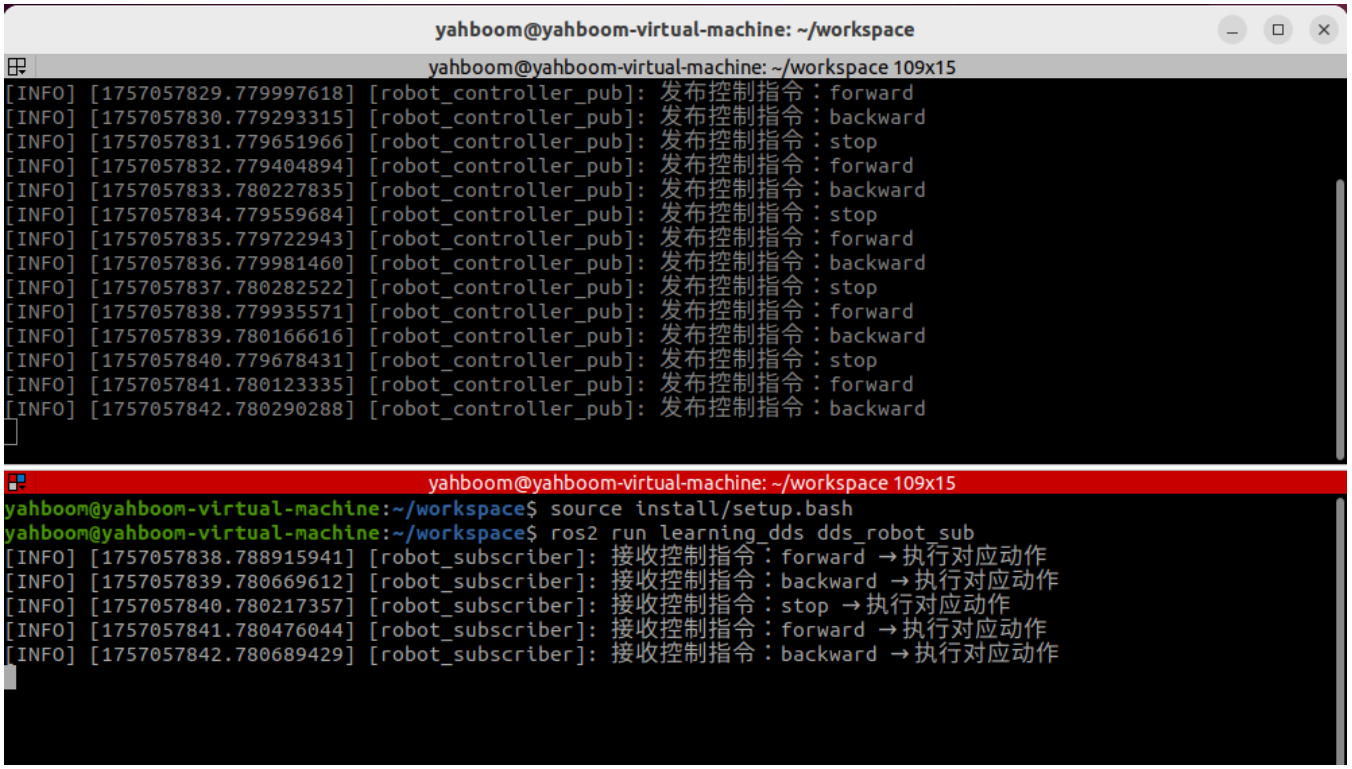
- Set the environment variables; they must be set every time you recompile

```
source install/setup.bash
```

- Run the publisher and subscriber nodes

```
ros2 run learning_dds dds_controller_pub
```

```
ros2 run learning_dds dds_robot_sub
```



Advanced robot development:

- Under normal circumstances, the publisher and the subscriber of a topic communication need to keep the same QoS communication policy, so as to avoid some implicit communication layer problems;
- The Qos communication policy should be selected according to the actual application scenario;
- If the application scenario involves drone communication, encrypted communication, real-time communication, or other scenarios with special requirements on the communication layer, find the official Fast-DDS manual in the introduction of this section for a more detailed description of the DDS features;

Fast DDS
3.3.0
Commercial support
Search
INTRODUCTION
What is *Fast DDS*?
Commercial support
Key features
DDS API
Fast DDS-Gen
RTPS Wire Protocol
Main Features
Dependencies and compatibilities

What is *Fast DDS*?

eProsima Fast DDS is a C++ implementation of the DDS (Data Distribution Service) Specification, a protocol defined by the Object Management Group (OMG). The *eProsima Fast DDS* library provides both an Application Programming Interface (API) and a communication protocol that deploy a Data-Centric Publisher-Subscriber (DCPS) model, with the purpose of establishing efficient and reliable information distribution among Real-Time Systems.

Commercial support

Looking for commercial support? Write us to info@eprosima.com

Find more about us at eProsima's webpage.

Key features